

1 Asymptotics is Fun!

- (a) Using the function `g` defined below, what is the runtime of the following function calls? Write each answer in terms of `N`.

```

1 void g(int N, int x) {
2     if (N == 0) {
3         return;
4     }
5     for (int i = 1; i <= x; i++) {
6         g(N - 1, i);
7     }
8 }

```

$g(N, 1) \rightarrow g(N-1, 1) \rightarrow g(N-2, 1) \dots g(0, 1)$
 $g(N, 1): \Theta(N)$
 $g(N, 2): \Theta(N^2)$

$g(N, 2) \rightarrow g(N-1, 1) \rightarrow g(N-2, 1)$
 $g(N-1, 2) \rightarrow g(N-2, 2)$

- (b) Suppose we change line 6 to `g(N - 1, x)` and change the stopping condition in the for loop to `i <= f(x)` where `f(x)` returns a random number between 1 and `x`, inclusive. For the following function calls, find the tightest Ω and big O bounds.

```

1 void g(int N, int x) {
2     if (N == 0) {
3         return;
4     }
5     for (int i = 1; i <= f(x); i++) {
6         g(N - 1, x);
7     }
8 }

```

$g(N, 2): \Omega(N), O(N^2)$
 $g(N, N): \Omega(N), O(N^N)$

$g(N, N) \rightarrow \begin{cases} g(N-1, N) \\ g(N-1, N-1) \\ g(N-1, N-2) \\ \dots \\ g(N-1, 1) \end{cases}$

$N \rightarrow \{ \}$

2 Flip Flop

Suppose we have the `flip` function as defined below. Assume the method `unknown` returns a random integer between 1 and N , exclusive, and runs in constant time. For each definition of the `flop` method below, give the best and worst case runtime of `flip` in $\Theta(\cdot)$ notation as a function of N .

```

1  public static void flip(int N) {
2      if (N <= 100) {
3          return;
4      }
5      int stop = unknown(N);
6      for (int i = 1; i < N; i++) {
7          if (i == stop) {
8              flop(i, N);
9              return;
10         }
11     }
12 }
```

(a) `public static void flop(int i, int N) {`
 `flip(N - i);`
`}`

Best Case: $\Theta(1)$, Worst Case: $\Theta(N)$

(b) `public static void flop(int i, int N) {`
 `int minimum = Math.min(i, N - i);`
 `flip(minimum);`
 `flip(minimum);`
`}`

$$\frac{N}{2} + \frac{N}{4} + \frac{N}{4} + \frac{N}{8} \times 4 + \frac{N}{16} \times 8$$

Best Case: $\Theta(1)$, Worst Case: $\Theta(N \log N)$

(c) `public static void flop(int i, int N) {`
 `flip(i);`
 `flip(N - i);`
`}`

everytime loop to `flop(N-1, N)`
 $\downarrow$
 2

Best Case: $\Theta(N)$, Worst Case: $\Theta(N^2)$

3 Prime Factors

Determine the best and worst case runtime of `prime_factors` in $\Theta(\cdot)$ notation as a function of N .

```

1  int prime_factors(int N) {
2      int factor = 2;
3      int count = 0;
4      while (factor * factor <= N) {
5          while (N % factor == 0) {
6              System.out.println(factor);
7              count += 1;
8              N = N / factor;
9          }
10         factor += 1;
11     }
12     return count;
13 }
```

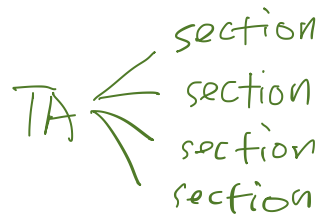
N is prime, factor += 1 until $\text{factor}^2 > N$

Best Case: $\Theta(\log N)$, Worst Case: $\Theta(\sqrt{N})$

meet 2 every time

4 ADT Selection

Suppose we have a TA Shreyas who teaches multiple discussion sections! A student may frequent more than one discussion section. For each situation below, choose the best ADT(s) out of the following — Map, Set, List — and explain how you can use the ADT(s) to solve the problem. Each subpart is independent of the previous. One answer may involve multiple ADTs. There may be multiple efficient answers for each problem.



1. Storing all the Students in Shreyas's first section in alphabetical order.

List[student]

2. Storing all the Students by their section, where Students within a section are sorted alphabetically.

List [List [student]]
section

3. Storing the Students in *all* of Shreyas's sections. There shouldn't be duplicates.

Set < student >

4. Determining all the Students that attend more than one of Shreyas's sections.

Map < student, count >

5. Quickly getting a Student by sid.

Map < sid, student >

6. Quickly getting a Student by name *or* sid. Names aren't necessarily unique.

Map < name, List < student > >
Map < sid, List < student > >

7. Cycling through the Students in one discussion section.

List < student >